DOTM — Dependency Origin & Traceability Module

DOTM — Dependency Origin & Traceability Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Dependency Standard (SDS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Dependency Origin & Traceability Governance

Module ID: SLGA-SDS-DOTM-002

Origin ID: OOF.CM.SLGA.03.02.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 11.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Dependency Origin & Traceability Module (DOTM) governs the

establishment, verification, preservation, and reassessment of origin, provenance, source relationships, and traceability for software dependencies upon which a software system materially relies.

DOTM establishes the methodological conditions under which the claimed

origin of a software dependency may be distinguished from its evidenced origin, and under which the dependency's movement through repositories, suppliers, registries, build processes, distribution mechanisms, transformations, and software integration paths may be reconstructed.

It governs the relationship between a dependency's identified component,

claimed source, originating entity or process, distribution path, transformation history, direct or transitive introduction path, relevant evidence, and the dependent software object.

DOTM distinguishes component identity from dependency origin, source claims

from supported provenance, and the existence of technical dependency records from evidence-supported traceability of the dependency relationship.

It applies to internal and external software components, packages,

libraries, frameworks, generated components, embedded resources, software services, development and build dependencies, platform resources, and material direct or transitive software dependency relationships.

Its principal output is:

Governed Dependency Origin & Traceability State

Its foundational distinction is:

Claimed Dependency Origin ≠ Verified Dependency Provenance.

Operational Role

The operational role of DOTM is:

To ensure that material software dependencies have sufficiently supported origin and traceability relationships so that their sources, introduction paths, transformations, distribution histories, and connections to dependent software objects can be reconstructed and reassessed throughout the software lifecycle.

DOTM establishes the origin and traceability foundation of SDS.

It answers the second dependency-governance question:

Where did this dependency come from, through which material paths did it reach the software system, and what evidence supports that history?

DOTM does not independently determine whether the dependency is necessary, suitable, authorized for inclusion, secure, or operationally sustainable.

It establishes the origin and traceability conditions required for

competent downstream dependency governance.

Core Question

How can the origin and material history of software dependencies remain reconstructable across suppliers, repositories, registries, build processes, transformations, transitive relationships, generated components, and autonomous development actions without treating source claims, package metadata, or recorded technical paths as sufficient proof of dependency provenance?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

DOTM operates within:

Software Dependency Standard (SDS)

The SDS internal governance flow is:

Component Identification → Dependency Origin & Traceability → Dependency

Suitability → Component Introduction & Change → Dependency Continuity & Risk

DOTM governs:

Dependency Origin & Traceability

It is the second Core Module of SDS.

Its principal upstream input is:

Governed Software Component Identification State

from:

CIM — Component Identification Module

Relevant software creation and artifact evidence may also be received from:

CRIM — Code & Artifact Integrity Module

SACM — Software Artifact Composition Module

DTM — Development Traceability Module

Its principal output is:

Governed Dependency Origin & Traceability State

This state supports:

DSM — Dependency Suitability Module

and the remaining SDS Core Modules.

DOTM may be invoked repeatedly when dependency origins, distribution paths, suppliers, versions, or transformation conditions materially change.

• Governed Space

Software Dependency Origin & Traceability Governance

This space governs the recognition, verification, preservation, and

continuity of material dependency origin and traceability relationships.

Its principal dimensions include:

• Dependency origin governance objects

• Dependency source identification

• Claimed origin

• Evidenced origin

• Origin attribution

• Publisher and supplier relationships

• Repository and registry relationships

• Distribution paths

• Artifact provenance

• Source-to-artifact relationships

• Build and transformation provenance

• Direct dependency introduction paths

• Transitive dependency paths

• Generated dependency provenance

• Embedded dependency provenance

• External service origin

• Dependency relationship traceability

• Origin evidence

• Traceability evidence

• Evidence competence

• Evidence integrity

• Origin uncertainty

• Provenance conflicts

• Traceability gaps

• Origin and path changes

• Provenance continuity

• Origin and traceability governance states

The governed space is independent of any specific package registry, source-control platform, programming language, supplier, software repository, or build technology.

Dependency Origin & Traceability Governance Object

Every DOTM assessment shall establish a Dependency Origin & Traceability

Governance Object.

The object represents the origin and traceability conditions of an

identified software dependency relationship.

It should identify:

Governance Object Identity: The identifiable origin and traceability

assessment.

Dependent Software Object: The software system relying upon the

dependency.

Dependency Component: The component identified by CIM.

Dependency Relationship: The material reliance relationship under

assessment.

Claimed Origin: The stated or reported source of the dependency.

Evidenced Origin: The origin supported by competent evidence.

Originating Entity or Process: The relevant producer, publisher,

generator, or originating mechanism.

Distribution Path: The relevant path through which the dependency was obtained.

Transformation Path: Material transformations affecting the dependency.

Introduction Path: The relationship through which the dependency entered the dependent software context.

Provenance Evidence: Evidence supporting the dependency's origin and

history.

Traceability Evidence: Evidence supporting material dependency

relationships.

Origin Uncertainty: Material unresolved origin conditions.

Traceability Gaps: Missing material relationship evidence.

Governance State: The current origin and traceability condition.

The governance object shall remain distinguishable from the component

itself.

Dependency Provenance Governance Object ≠ Software Component Artifact.

Dependency Origin Identification

DOTM shall identify the claimed and evidenced origin of material software

dependencies.

Origin information may include:

• Original producer

• Publishing entity

• Source repository

• Package registry

• Internal software team

• External software supplier

• Software generation mechanism

• Source distribution

• Relevant artifact source

DOTM shall distinguish a source reference from evidence that the source genuinely originated the dependency.

Source Reference ≠ Verified Dependency Origin.

Claimed Dependency Origin

A Claimed Dependency Origin is an assertion identifying where a dependency allegedly originated.

Claims may appear in:

• Package metadata

• Supplier documentation

• Repository descriptions

• Software inventories

• Build records

• Developer statements

• AI-generated explanations

• Distribution manifests

DOTM shall preserve claims as claims until competent evidence supports them.

Claimed Dependency Origin ≠ Verified Dependency Provenance.

Evidenced Dependency Origin

An Evidenced Dependency Origin is an origin relationship supported by relevant and competent evidence.

Supporting evidence may include:

• Authenticated publication records

• Repository history

• Signed artifact attestations

• Verified build provenance

• Supplier records

• Content integrity evidence

• Consistent distribution records

• Relevant origin-control records

DOTM shall determine whether the available evidence supports the particular

origin claim being assessed.

An evidenced origin may remain incomplete or subject to material limitations.

Evidence of Publication ≠ Proof of Original Creation.

Origin Attribution

DOTM shall identify the entity or mechanism to which a material dependency

origin is attributed.

Relevant actors may include:

• Individual developers

• Development organizations

• Software suppliers

• Open-source projects

• Automated build systems

• Code generation systems

• AI development agents

• Autonomous software creation systems

Attribution shall distinguish the entity that created a component from entities that published, distributed, modified, or integrated it.

Component Creator ≠ Component Publisher ≠ Component Distributor.

Origin Authority Reference

DOTM shall preserve relevant references concerning the authority under

which a component was created, published, or distributed where material to its origin.

It shall not infer legitimate authority solely from technical publication

capability.

Detailed authority determination remains within competent authority and dependency introduction governance.

Publication Capability ≠ Legitimate Publication Authority.

Source Repository Identification

DOTM shall identify material source repositories associated with software

dependencies.

Relevant information may include:

• Repository identity

• Repository location

• Relevant revision

• Branch or tag reference

• Source content identifier

• Relevant historical records

A repository address does not independently establish that the delivered dependency artifact originated from that repository state.

Repository Reference ≠ Verified Artifact Provenance.

Package Registry Identification

DOTM shall identify material package registries and distribution mechanisms

associated with dependencies.

Relevant information may include:

• Registry identity

• Package namespace

• Published package identity

• Version

• Distribution reference

• Publisher claim

• Relevant publication evidence

Registry presence shall not independently establish original authorship or

component authenticity.

Registry Publication ≠ Verified Original Authorship.

Supplier Origin Relationships

DOTM shall identify material supplier relationships affecting dependency

origin.

A dependency may pass through multiple entities before reaching the dependent software system.

Relevant roles may include:

• Original producer

• Publisher

• Distributor

• Reseller

• Integrator

• Internal supplier

• External supplier

• Maintenance provider

DOTM shall preserve distinctions between these roles.

Supplier Identity ≠ Original Component Origin.

Dependency Distribution Path

DOTM shall establish material distribution paths through which software dependencies reach the dependent software environment.

A path may be:

Producer → Repository → Registry → Package Manager → Build Environment →

Software Artifact

Alternative paths may involve direct downloads, internal mirrors, supplier delivery, or generated artifacts.

DOTM shall preserve material intermediaries where evidence permits.

Known Distribution Endpoint ≠ Complete Distribution Path.

Direct Dependency Introduction Traceability

DOTM shall establish how directly relied-upon components entered the

dependent software context.

Relevant paths may include:

• Developer selection

• Automated dependency installation

• AI agent introduction

• Supplier integration

• Build configuration

• Runtime service configuration

• Manual component embedding

DOTM governs traceability of the introduction path.

Legitimate authorization of introduction remains within CICM.

Recorded Dependency Introduction ≠ Authorized Dependency Introduction.

Transitive Dependency Traceability

DOTM shall preserve material transitive dependency relationships.

A relevant chain may be:

Application → Direct Library → Transitive Library → Supporting Component

Traceability shall preserve the intermediate relationships where sufficiently evidenced.

An indirectly introduced dependency shall not be treated as originless

merely because the dependent application did not select it directly.

Transitive Introduction ≠ Absence of Traceable Origin.

Multi-Path Dependency Traceability

A component may reach a software system through multiple dependency paths.

DOTM shall preserve materially distinct paths where they affect origin or traceability.

For example:

Application → Framework A → Library X

and:

Application → Framework B → Library X

The same component identity may appear through multiple relationships.

DOTM shall distinguish component identity from path identity.

Shared Component Identity ≠ Shared Dependency Path.

Dependency Path Completeness

DOTM shall assess whether material dependency introduction paths are sufficiently reconstructable.

Completeness may concern:

• Source identification

• Distribution intermediaries

• Transformation stages

• Direct introduction

• Transitive introduction

• Runtime resolution

• Final integration

• Supporting evidence

Missing material stages shall remain explicit.

Known Origin and Destination ≠ Complete Dependency Traceability.

Artifact Provenance

DOTM shall establish relevant provenance relationships between software

dependencies and their distributed artifacts.

Relevant evidence may include:

• Artifact identifiers

• Content hashes

• Signed attestations

• Build references

• Publication records

• Distribution records

• Version relationships

Artifact integrity evidence may support provenance but shall not independently establish the complete historical origin.

Artifact Integrity ≠ Complete Artifact Provenance.

Source-to-Artifact Traceability

DOTM shall preserve material relationships between source references and

resulting dependency artifacts.

A relevant chain may be:

Source Revision → Build Process → Build Artifact → Distributed Package

Where this relationship cannot be established, the gap shall remain explicit.

Source Availability ≠ Verified Source-to-Artifact Correspondence.

Build Provenance

DOTM shall identify material build processes affecting dependency

provenance.

Relevant information may include:

• Build system

• Build environment

• Source inputs

• Build instructions

• Toolchain references

• Generated outputs

• Artifact identifiers

• Build evidence

DOTM shall distinguish a recorded build process from evidence that the distributed artifact was produced through that process.

Build Record Existence ≠ Verified Build Provenance.

Dependency Transformation Traceability

DOTM shall preserve material transformations affecting software

dependencies.

Transformations may include:

• Compilation

• Bundling

• Minification

• Packaging

• Patching

• Rebuilding

• Reconfiguration

• Code generation

• Component merging

• Vendor modification

A transformed component may retain a relationship to its source while differing materially from the original artifact.

Source Component Identity ≠ Unchanged Distributed Artifact.

Repackaged Dependency Provenance

DOTM shall identify material dependencies repackaged or redistributed

through different mechanisms.

Examples include:

• Internal package mirrors

• Vendor-specific distributions

• Repackaged open-source libraries

• Customized software distributions

• Bundled dependency archives

DOTM shall distinguish the original component origin from the repackaging

or redistribution event.

Repackaging Origin ≠ Original Component Origin.

Forked Dependency Provenance

DOTM shall identify material fork relationships where a dependency derives from an earlier software project or component.

Relevant information may include:

• Original project reference

• Fork identity

• Forking event

• Relevant source relationship

• Material modifications

• Distribution identity

A fork shall not automatically be treated as identical to its source.

Common Source History ≠ Equivalent Dependency Provenance.

Embedded Dependency Provenance

DOTM shall govern origin and traceability for dependencies embedded within larger software artifacts.

Embedded components may lack independent package metadata.

Relevant evidence may include:

• Source inclusion records

• Build inputs

• Supplier declarations

• Component composition records

• Artifact analysis

• Embedded version references

A complete provenance claim for a composite artifact shall not

automatically establish provenance for every embedded component.

Composite Artifact Provenance ≠ Complete Constituent Provenance.

Generated Dependency Provenance

DOTM shall identify material software dependencies created through automated generation mechanisms.

Relevant mechanisms may include:

• Code generators

• AI coding assistants

• Autonomous software development agents

• Automated interface generation

• Model-based code generation

• Build-time component generation

DOTM shall preserve relevant generation relationships and evidence.

A generated component may have multiple origin dimensions, including source inputs, generation instructions, models, tools, and transformation processes.

Generated Artifact Existence ≠ Reconstructable Generation Provenance.

AI-Generated Dependency Origin

Where AI systems materially participate in dependency creation, DOTM shall distinguish:

• The AI system used

• The generation context

• Material source inputs

• Generated artifact identity

• Relevant transformation steps

• The entity or process publishing the artifact

• The entity or process introducing the artifact into the dependent

software.

An AI system's output shall not automatically establish original

authorship, lawful provenance, or verified source correspondence.

AI Generation Attribution ≠ Complete Dependency Origin Verification.

Autonomous Dependency Introduction Traceability

Where an autonomous development agent introduces a dependency, DOTM shall preserve the material introduction path.

A relevant path may be:

Development Objective → Agent Action → Dependency Selection → Retrieval

Source → Retrieved Artifact → Integration Action → Dependent Software

DOTM governs traceability of the material path.

Authority to execute the objective does not independently establish authority for each constituent action.

Task Authority ≠ Constituent Action Authority.

Authorization assessment remains within competent authority governance and CICM.

External Service Origin

DOTM shall establish relevant origin relationships for external software

services.

Relevant information may include:

• Service provider identity

• Service ownership reference

• Service publication or provision mechanism

• Interface identity

• Material service version

• Relevant service provenance evidence

An endpoint address shall not independently establish service origin or

operator identity.

Service Endpoint Identity ≠ Verified Service Provider Origin.

Platform Dependency Origin

DOTM shall identify material origin and distribution relationships for

software platform dependencies.

These may include:

• Runtime platforms

• Operating systems

• Container images

• Application servers

• Managed execution environments

• Database platforms

Platform provenance may involve multiple software layers.

DOTM shall preserve material layer relationships where relevant.

Platform Label ≠ Complete Platform Dependency Provenance.

Dependency Version Provenance

DOTM shall establish the origin and traceability of relevant dependency

versions.

Version provenance may include:

• Source revision

• Release reference

• Build reference

• Distribution artifact

• Publication event

• Relevant content identifier

A version label shall not independently establish correspondence with a

specific source or artifact.

Version Label ≠ Verified Version Provenance.

Dependency Provenance Evidence

DOTM shall preserve evidence supporting material origin and traceability

claims.

Evidence may include:

• Repository histories

• Source revision records

• Publication records

• Signed attestations

• Artifact identifiers

• Build records

• Supplier documentation

• Distribution logs

• Component inventories

• Runtime observations

• Development traceability records

Evidence shall remain associated with the specific claims it supports.

Provenance Evidence Presence ≠ Provenance Claim Validity.

Provenance Evidence Competence

DOTM shall assess whether evidence is competent to support the origin or traceability claim.

For example:

A package registry record may support publication through a registry.

A repository history may support source development history.

A build attestation may support a particular build claim.

A cryptographic signature may support a signer-to-artifact relationship.

A runtime observation may support actual component use.

These records do not establish identical facts.

Evidence of One Provenance Stage ≠ Evidence of Every Provenance Stage.

Provenance Evidence Integrity

DOTM shall preserve the integrity of material provenance evidence.

Relevant considerations may include:

• Evidence source

• Content integrity

• Signature validity

• Temporal relevance

• Chain of custody

• Record preservation

• Material contradictions

• Evidence completeness

Evidence integrity does not independently establish that the underlying provenance claim is true.

Cryptographically Intact Evidence ≠ Complete Provenance Truth.

Provenance Claim Verification

DOTM shall determine whether material provenance claims are sufficiently supported.

Verification should consider:

• Claim specificity

• Relevant origin object

• Evidence competence

• Evidence integrity

• Source correspondence

• Artifact correspondence

• Distribution continuity

• Transformation history

• Material contradictions

• Remaining uncertainty

A verified claim shall remain limited to the facts supported by evidence.

Verified Provenance Claim ≠ Unrestricted Trust in the Dependency.

Dependency Traceability Reconstruction

DOTM shall support reconstruction of material dependency origin and

introduction paths.

A reconstructed path may include:

Origin → Source → Build → Artifact → Distribution → Introduction →

Dependent Software

Reconstruction shall distinguish:

• Directly evidenced stages

• Inferred stages

• Claimed stages

• Missing stages

• Conflicted stages

Inferred relationships shall not be represented as directly verified

evidence.

Reconstructed Dependency Path ≠ Fully Verified Dependency Path.

Origin Uncertainty

Origin uncertainty may arise where:

• The original producer is unknown

• Publication records are incomplete

• Repository ownership is ambiguous

• A component has multiple competing origin claims

• The delivered artifact cannot be linked to a source revision

• A generated component lacks sufficient generation records

• A supplier cannot establish the origin of embedded software

DOTM shall preserve material uncertainty.

Unknown Origin ≠ Proven Malicious Origin.

Provenance Conflict

A provenance conflict exists where material evidence supports incompatible origin or traceability claims.

Examples may include:

• Conflicting publisher identities

• Inconsistent source histories

• Contradictory build attestations

• Different artifact identities for the same claimed release

• Incompatible distribution records

• Conflicting generation claims

DOTM shall preserve the conflict and its supporting evidence.

Conflicting Provenance Claims ≠ Permission to Select the Convenient Claim.

Traceability Gap

A Dependency Traceability Gap exists where a material stage or relationship in a dependency's origin or introduction path cannot be sufficiently reconstructed.

Examples may include:

• Missing build provenance

• Unknown distribution intermediary

• Untraceable package replacement

• Unrecorded dependency transformation

• Missing transitive dependency introduction path

• Incomplete AI generation history

DOTM shall identify the gap and its material significance.

Traceability Gap ≠ Proven Dependency Compromise.

Critical Provenance Gap

A Critical Provenance Gap exists where insufficient origin or traceability evidence prevents competent assessment of a materially consequential dependency.

Examples may include:

• An unidentified source for a critical runtime component

• A dependency artifact whose origin cannot be established

• A supplier-delivered critical library with conflicting provenance

• An autonomous agent-introduced component lacking reconstructable

retrieval and integration evidence.

DOTM shall preserve the critical gap for competent downstream assessment.

It does not independently determine software suspension, security failure, or legal liability.

Critical Provenance Gap ≠ Automatic Security Incident.

Dependency Origin Drift

Dependency Origin Drift occurs when the material origin or distribution relationships of a dependency change relative to its recognized provenance state.

Possible causes include:

• Publisher changes

• Repository transfers

• Package ownership changes

• Distribution channel changes

• Supplier replacement

• Build system changes

• Artifact substitution

• Source project forks

• Autonomous dependency retrieval changes

DOTM shall identify material drift and support reassessment.

Stable Package Name ≠ Stable Dependency Origin.

Provenance Change Recognition

DOTM shall preserve material changes in dependency provenance.

Changes may include:

• New source references

• Corrected origin attribution

• Publisher changes

• New distribution paths

• Revised build provenance

• Component transformations

• Resolved provenance conflicts

• Newly discovered traceability gaps

Historical provenance states shall remain reconstructable where material.

Provenance Correction ≠ Historical Provenance Erasure.

Provenance Continuity

DOTM shall preserve material provenance relationships across software

lifecycle changes.

Continuity may involve:

• Dependency updates

• Package migrations

• Supplier transitions

• Repository changes

• Software modernization

• Build system replacement

• Runtime platform changes

• Component repackaging

• Component forks

A dependency's name or functional role may remain unchanged while its provenance changes materially.

Functional Continuity ≠ Provenance Continuity.

Dependency Origin & Traceability Governance States

DOTM shall support identifiable governance states.

Possible states include:

Origin-Claimed: A dependency origin has been asserted but not

sufficiently evidenced.

Under Assessment: Origin and traceability conditions are being evaluated.

Origin-Supported: Material origin claims are supported by competent evidence within the assessed scope.

Traceability-Supported: Material dependency paths are sufficiently reconstructable within the assessed scope.

Conditionally Supported: Origin or traceability recognition depends on

explicit conditions.

Partially Supported: Some material provenance dimensions remain

unresolved.

Evidence-Limited: Available evidence is insufficient for the required determination.

Origin-Unresolved: Material origin conditions cannot be sufficiently

established.

Path-Unresolved: A material dependency path cannot be reconstructed.

Transformation-Unresolved: Material dependency transformation history

remains uncertain.

Conflicted: Material provenance evidence is contradictory.

Continuity-at-Risk: Material provenance continuity cannot be sufficiently

maintained.

Superseded: A later governance state has replaced the earlier state.

Unresolved: Available information does not support a sufficiently determined state.

These states shall not be interpreted as dependency suitability, security,

or acceptance determinations.

Provenance Supported ≠ Dependency Accepted.

Origin & Traceability Recognition

DOTM shall determine whether the material origin and traceability

conditions of a dependency are sufficiently supported for the intended governance purpose.

Recognition should consider:

• Component identity

• Claimed origin

• Evidenced origin

• Origin attribution

• Source relationships

• Publisher relationships

• Distribution paths

• Transformation history

• Direct and transitive introduction paths

• Artifact provenance

• Evidence competence

• Evidence integrity

• Material uncertainty

• Conflicts

• Traceability gaps

• Provenance continuity

Recognition shall remain limited to the provenance relationships actually

assessed.

Recognized Provenance ≠ Complete Dependency Governance.

Dependency Origin and Suitability

DOTM establishes origin and traceability conditions.

It does not independently determine whether a dependency is necessary or suitable.

Detailed suitability governance remains within:

DSM — Dependency Suitability Module

A dependency may have fully supported provenance while being unsuitable for the intended software context.

Verified Origin ≠ Dependency Suitability.

Dependency Origin and Introduction Authority

DOTM preserves material dependency introduction paths.

It does not independently establish legitimate authority to introduce or change a dependency.

Detailed introduction and change governance remains within:

CICM — Component Introduction & Change Module

A dependency may be traceable to a known source while having been introduced without authorization.

Traceable Introduction ≠ Authorized Introduction.

Dependency Origin and Continuity

DOTM preserves provenance continuity and identifies material changes in

origin relationships.

It does not independently establish maintenance, availability, substitutability, or operational sustainability.

Detailed continuity and risk governance remains within:

DCRM — Dependency Continuity & Risk Module

Provenance Continuity ≠ Operational Dependency Continuity.

Dependency Origin Reassessment

DOTM shall support reassessment when material provenance conditions change.

Triggers may include:

• New origin evidence

• Publisher changes

• Repository transfers

• Dependency updates

• Artifact substitutions

• Supplier changes

• Build system changes

• New distribution paths

• Newly discovered transitive dependencies

• Autonomous agent dependency introduction

• Provenance conflicts

• Traceability gaps

• Origin drift

• Software lifecycle changes

Reassessment shall preserve material historical provenance records.

Governed Dependency Origin & Traceability State

The principal output of DOTM is:

Governed Dependency Origin & Traceability State

The state shall preserve sufficient information to identify:

Origin Governance Object: The dependency origin and traceability

assessment.

Dependent Software Object: The software system relying on the dependency.

Component Identity: The component identified by CIM.

Dependency Relationship: The relevant material reliance relationship.

Claimed Origin: The stated origin of the dependency.

Evidenced Origin: The origin supported by competent evidence.

Origin Attribution: The relevant producer, publisher, supplier, or

generating mechanism.

Source Reference: The relevant source repository or source object.

Publication Reference: The relevant publication or registry relationship.

Distribution Path: The material dependency distribution route.

Transformation Path: Material modifications affecting the dependency.

Introduction Path: The route through which the dependency entered the dependent software.

Direct or Transitive Path: The relevant dependency relationship structure.

Artifact Provenance: The relationship between source, build, and

distributed artifact.

Generation Provenance: Relevant generation evidence where applicable.

Origin Evidence: Evidence supporting origin claims.

Traceability Evidence: Evidence supporting dependency paths.

Evidence Competence: The suitability of evidence for the claim assessed.

Evidence Integrity: The condition of supporting evidence.

Origin Uncertainty: Material unresolved origin conditions.

Provenance Conflicts: Contradictory origin evidence.

Traceability Gaps: Missing material dependency path evidence.

Critical Provenance Gaps: Material provenance failures requiring

competent attention.

Origin Drift: Changes in recognized origin or distribution conditions.

Provenance Continuity: Preservation of material origin relationships

across change.

Origin & Traceability Governance State: The recognized governance

condition.

Downstream References: Information required by competent SDS governance

modules.

The state shall not independently establish dependency suitability,

introduction authority, security, quality, or operational continuity.

Minimum Implementation Framework

A conforming implementation of DOTM must provide the following six functional capabilities.

• Dependency Origin Object and Source Governance

Capability to establish dependency origin governance objects, identify

claimed sources, and distinguish original producers, publishers, suppliers, distributors, and generating mechanisms.

The implementation must preserve claimed origins separately from evidenced origins.

• Source, Publication, Distribution, and Transformation Traceability

Governance

Capability to reconstruct material source-to-artifact relationships,

publication paths, distribution intermediaries, build provenance, repackaging, forks, and component transformations.

The implementation must preserve missing or uncertain stages rather than treating an incomplete chain as verified.

• Direct, Transitive, Generated, and Autonomous Dependency Path Governance

Capability to establish material dependency introduction paths across

direct, transitive, embedded, generated, dynamically resolved, and autonomous development contexts.

The implementation must distinguish technical introduction traceability from legitimate introduction authority.

• Provenance Evidence, Competence, Integrity, and Verification Governance

Capability to associate material origin and traceability claims with

competent evidence, preserve evidence integrity, and determine which claims are sufficiently supported.

The implementation must distinguish the specific provenance facts established by each evidence type.

• Origin Gap, Conflict, Drift, and Continuity Governance

Capability to identify material provenance gaps, critical gaps,

contradictory origin claims, distribution changes, artifact substitutions, and provenance continuity conditions.

The implementation must preserve material historical provenance states and corrections.

• Governed Dependency Origin & Traceability State and SDS Handoff

Governance

Capability to produce a Governed Dependency Origin & Traceability State

with sufficient source, origin, distribution, transformation, introduction path, evidence, uncertainty, and continuity information for competent downstream SDS governance.

The implementation must not convert supported provenance into dependency acceptance.

GIE™ Compatibility

DOTM is compatible with GIE™ as a downstream methodological analysis and execution layer.

GIE™ may support:

• Dependency source discovery

• Claimed origin extraction

• Origin evidence association

• Producer and publisher distinction

• Supplier relationship mapping

• Repository history analysis

• Package registry relationship analysis

• Source-to-artifact traceability

• Build provenance analysis

• Distribution path reconstruction

• Transformation path reconstruction

• Repackaged dependency recognition

• Fork relationship analysis

• Embedded component provenance analysis

• Generated component provenance analysis

• Autonomous agent dependency path reconstruction

• External service origin assessment

• Platform provenance assessment

• Version provenance correlation

• Provenance evidence competence assessment

• Provenance evidence integrity assessment

• Origin claim verification support

• Traceability gap detection

• Critical provenance gap detection

• Provenance conflict analysis

• Origin drift detection

• Provenance continuity assessment

• Governed provenance state representation

GIE™ may infer candidate provenance relationships or identify possible origin conflicts.

Such outputs shall remain distinguishable from sufficiently evidenced

provenance relationships.

GIE™ must not fabricate source histories, invent distribution paths, infer legitimate origin authority from publication capability, or convert incomplete provenance into verified origin.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical DOTM governance logic is:

Receive Governed Software Component Identification State

↓

Identify Dependency Origin & Traceability Governance Object

↓

Identify Dependent Software Object

↓

Identify Component Identity and Dependency Relationship

↓

Identify Claimed Dependency Origin

↓

Identify Originating Entity or Mechanism

↓

Identify Source Repository or Source Object

↓

Identify Publisher and Supplier Relationships

↓

Identify Package Registry and Distribution References

↓

Identify Source-to-Artifact Relationships

↓

Identify Build and Transformation Paths

↓

Identify Direct Dependency Introduction Path

↓

Identify Transitive and Multi-Path Dependency Relationships

↓

Identify Generated, Embedded, Service, and Platform Provenance

↓

Identify Autonomous Dependency Introduction Path Where Applicable

↓

Associate Origin and Traceability Evidence

↓

Assess Evidence Competence and Integrity

↓

Assess Origin Claim Support

↓

Assess Distribution and Transformation Traceability

↓

Assess Dependency Path Completeness

↓

Identify Origin Uncertainty

↓

Identify Provenance Conflicts

↓

Identify Traceability Gaps and Critical Provenance Gaps

↓

Identify Origin Drift

↓

Determine Origin & Traceability Governance State

↓

Establish Governed Dependency Origin & Traceability State

↓

Preserve Provenance Continuity

↓

Handoff to DSM and Competent SDS Governance

↓

Reassess When Material Provenance Conditions Change

The principal output is:

Governed Dependency Origin & Traceability State

Governance Outputs

DOTM may produce:

Dependency Origin & Traceability Governance Object

Dependency Origin Identification Record

Claimed Dependency Origin Record

Evidenced Dependency Origin Record

Origin Attribution Record

Origin Authority Reference

Source Repository Identification Record

Package Registry Identification Record

Supplier Origin Relationship Record

Dependency Distribution Path

Direct Dependency Introduction Trace

Transitive Dependency Trace

Multi-Path Dependency Trace

Dependency Path Completeness Assessment

Artifact Provenance Record

Source-to-Artifact Traceability Record

Build Provenance Record

Dependency Transformation Trace

Repackaged Dependency Provenance Record

Forked Dependency Provenance Record

Embedded Dependency Provenance Record

Generated Dependency Provenance Record

AI-Generated Dependency Origin Record

Autonomous Dependency Introduction Trace

External Service Origin Record

Platform Dependency Origin Record

Dependency Version Provenance Record

Dependency Provenance Evidence Record

Provenance Evidence Competence Assessment

Provenance Evidence Integrity Assessment

Provenance Claim Verification Record

Dependency Traceability Reconstruction

Origin Uncertainty State

Provenance Conflict Record

Dependency Traceability Gap

Critical Provenance Gap

Dependency Origin Drift Record

Provenance Change Record

Provenance Continuity Record

Origin & Traceability Governance State

Origin & Traceability Recognition Record

Dependency Suitability Reference

Dependency Introduction Authority Reference

Dependency Continuity Reference

Dependency Origin Reassessment Record

SDS Provenance Handoff Record

The principal output is:

Governed Dependency Origin & Traceability State

Critical Distinctions

DOTM preserves the following distinctions:

Claimed Dependency Origin ≠ Verified Dependency Provenance

Dependency Provenance Governance Object ≠ Software Component Artifact

Source Reference ≠ Verified Dependency Origin

Evidence of Publication ≠ Proof of Original Creation

Component Creator ≠ Component Publisher ≠ Component Distributor

Publication Capability ≠ Legitimate Publication Authority

Repository Reference ≠ Verified Artifact Provenance

Registry Publication ≠ Verified Original Authorship

Supplier Identity ≠ Original Component Origin

Known Distribution Endpoint ≠ Complete Distribution Path

Recorded Dependency Introduction ≠ Authorized Dependency Introduction

Transitive Introduction ≠ Absence of Traceable Origin

Shared Component Identity ≠ Shared Dependency Path

Known Origin and Destination ≠ Complete Dependency Traceability

Artifact Integrity ≠ Complete Artifact Provenance

Source Availability ≠ Verified Source-to-Artifact Correspondence

Build Record Existence ≠ Verified Build Provenance

Source Component Identity ≠ Unchanged Distributed Artifact

Repackaging Origin ≠ Original Component Origin

Common Source History ≠ Equivalent Dependency Provenance

Composite Artifact Provenance ≠ Complete Constituent Provenance

Generated Artifact Existence ≠ Reconstructable Generation Provenance

AI Generation Attribution ≠ Complete Dependency Origin Verification

Task Authority ≠ Constituent Action Authority

Service Endpoint Identity ≠ Verified Service Provider Origin

Platform Label ≠ Complete Platform Dependency Provenance

Version Label ≠ Verified Version Provenance

Provenance Evidence Presence ≠ Provenance Claim Validity

Evidence of One Provenance Stage ≠ Evidence of Every Provenance Stage

Cryptographically Intact Evidence ≠ Complete Provenance Truth

Verified Provenance Claim ≠ Unrestricted Trust in the Dependency

Reconstructed Dependency Path ≠ Fully Verified Dependency Path

Unknown Origin ≠ Proven Malicious Origin

Conflicting Provenance Claims ≠ Permission to Select the Convenient Claim

Traceability Gap ≠ Proven Dependency Compromise

Critical Provenance Gap ≠ Automatic Security Incident

Stable Package Name ≠ Stable Dependency Origin

Provenance Correction ≠ Historical Provenance Erasure

Functional Continuity ≠ Provenance Continuity

Provenance Supported ≠ Dependency Accepted

Recognized Provenance ≠ Complete Dependency Governance

Verified Origin ≠ Dependency Suitability

Traceable Introduction ≠ Authorized Introduction

Provenance Continuity ≠ Operational Dependency Continuity

Cross-Architecture Boundary

DOTM governs dependency origin and traceability within SDS and SLGA™.

It does not independently establish dependency necessity, suitability, introduction authority, security, quality, or operational continuity.

Within SDS:

CIM — Component Identification Module governs component identification.

DOTM — Dependency Origin & Traceability Module governs origin,

provenance, and traceability.

DSM — Dependency Suitability Module governs necessity, compatibility, and

suitability.

CICM — Component Introduction & Change Module governs introduction

authority, inclusion, modification, replacement, and removal.

DCRM — Dependency Continuity & Risk Module governs criticality, risks,

maintenance, availability, substitutability, and continuity.

Within SLGA™:

Software Requirements Standard (SRS) governs requirements.

Software Creation Standard (SCS) governs software creation and artifact

formation.

Software Configuration Standard (SCFS) governs software configuration.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs release.

Software Deployment Standard (SDPS) governs deployment.

Software Change Management Standard (SCMS) governs broader lifecycle

change.

Software Retirement Standard (SRTS) governs retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures include:

OBIDENITY® — Origin-Bound Identity Governance Architecture: Relevant

origin and identity governance.

GOA™ — Governance Architecture: General authority governance.

ORA™ — Operational Reality Architecture: Competent operational reality

assessment.

AGA™ — Accountability Governance Architecture: Relevant accountability

relationships.

DOTM preserves provenance relationships without replacing the competent

governance responsibilities of these domains.

Canonical Boundary

DOTM begins when a material software component or dependency resource has been identified sufficiently to establish an origin and traceability assessment.

DOTM governs:

• Dependency origin governance objects

• Claimed origins

• Evidenced origins

• Origin attribution

• Origin authority references

• Source repositories

• Package registries

• Supplier origin relationships

• Distribution paths

• Direct introduction paths

• Transitive introduction paths

• Multi-path dependency relationships

• Dependency path completeness

• Artifact provenance

• Source-to-artifact traceability

• Build provenance

• Dependency transformations

• Repackaged dependencies

• Forked dependencies

• Embedded dependency provenance

• Generated dependency provenance

• AI-generated dependency origin

• Autonomous dependency introduction traceability

• External service origin

• Platform dependency origin

• Version provenance

• Provenance evidence

• Evidence competence

• Evidence integrity

• Provenance claim verification

• Traceability reconstruction

• Origin uncertainty

• Provenance conflicts

• Traceability gaps

• Critical provenance gaps

• Origin drift

• Provenance changes

• Provenance continuity

• Origin and traceability governance states

• Origin and traceability recognition

• Dependency origin reassessment

DOTM does not independently determine:

• Component identity completeness

• Dependency necessity

• Dependency suitability

• Dependency introduction authorization

• Dependency acceptance

• Software security

• Software quality

• Software functional correctness

• Operational dependency continuity

• Software release readiness

• Legal liability

DOTM ends when the relevant dependency origin and traceability relationships have been represented as a:

Governed Dependency Origin & Traceability State

with sufficient origin, source, distribution, transformation, introduction

path, evidence, uncertainty, and continuity information for competent downstream SDS governance.

Use Case 1 — A Trusted Package Name Conceals a Changed Distribution

Origin

An organization operates a software platform that depends on a widely used external package.

The package has been included in the software for several years.

Its name remains unchanged.

Its version numbers continue increasing normally.

Automated dependency updates continue retrieving new releases.

The software compiles successfully and passes its functional tests.

However, the package's publishing control has changed.

A new entity now distributes releases under the same package identity.

The organization's dependency records identify the package name and version

but do not preserve sufficient evidence of the change in publishing origin.

The technical dependency appears continuous.

Its provenance is not.

DOTM evaluates:

Component Identity → Claimed Origin → Publisher Relationship →

Distribution Path → Artifact Provenance → Origin Drift

The assessment identifies:

• The component name remained unchanged

• The distribution origin changed

• Historical provenance records no longer fully describe the current

• Successful installation does not establish continuity of origin

• The new distribution relationship requires evidence-supported

dependency. reassessment.

The relevant governance state may be:

Origin-Unresolved

or:

Partially Supported

until the material origin relationship is sufficiently established.

The critical distinctions are:

Stable Package Name ≠ Stable Dependency Origin.

Registry Publication ≠ Verified Original Authorship.

Functional Continuity ≠ Provenance Continuity.

The result: DOTM exposes a change in dependency origin that conventional package-name and version tracking may fail to recognize.

Use Case 2 — An Autonomous AI Agent Introduces a Library Without

Reconstructable Provenance

An organization authorizes an autonomous AI development agent to implement

a software feature.

The agent searches for a suitable external library.

It retrieves a software package from an online distribution source.

The agent integrates the package, modifies part of its code, and

incorporates the result into the application.

The application compiles successfully.

Functional tests pass.

The development task is marked complete.

Later, the organization attempts to reconstruct the origin of the

incorporated library.

The available development records show the resulting software artifact but

do not establish:

• The original retrieval source

• The exact downloaded artifact

• The relevant source revision

• The transformations performed by the agent

• The relationship between the downloaded package and the final

incorporated component.

The organization cannot competently determine whether the final component

corresponds to the claimed source.

DOTM evaluates:

Development Objective → Agent Retrieval → Source Reference → Retrieved

Artifact → Transformation → Integrated Component → Dependent Software

The assessment identifies:

• The resulting component is present

• The claimed source cannot be sufficiently connected to the final

• Material transformation history is missing

• The introduction path is not fully reconstructable

• The provenance evidence is insufficient for the claimed origin

artifact. relationship.

The relevant governance state may be:

Transformation-Unresolved

or:

Evidence-Limited

until sufficient evidence is established.

The critical distinctions are:

Generated Artifact Existence ≠ Reconstructable Generation Provenance.

Source Availability ≠ Verified Source-to-Artifact Correspondence.

Reconstructed Dependency Path ≠ Fully Verified Dependency Path.

Task Authority ≠ Constituent Action Authority.

The result: DOTM reveals that successful autonomous software creation does not establish the provenance of the software dependencies introduced during that creation.

Canonical Closing Statement

Software dependencies do not appear without a history.

They are created, published, distributed, transformed, selected, integrated, updated, and sometimes replaced.

Each of these events may change what a software system actually relies upon.

A dependency may retain the same name while its publisher changes.

A package may carry a valid signature without proving where its source code originated.

A supplier may deliver a functioning component without establishing the history of its embedded libraries.

An autonomous AI agent may create working software while leaving no sufficiently reconstructable record of the external components it retrieved and transformed.

In each case, the dependency exists.

The software may function.

But the origin and material history of that dependency may remain unresolved.

Claimed Dependency Origin ≠ Verified Dependency Provenance.

Repository Reference ≠ Verified Artifact Provenance.

Source Availability ≠ Verified Source-to-Artifact Correspondence.

Traceable Introduction ≠ Authorized Introduction.

Functional Continuity ≠ Provenance Continuity.

DOTM establishes a fundamental requirement:

A software dependency must not be considered provenance-governed merely because its name, source address, publisher claim, or distribution record is available. Its material origin and introduction relationships must be supported by evidence sufficient for the governance purpose.

This requirement applies to software created by humans, organizations,

automated tools, AI coding assistants, and autonomous development agents.

The actor may change.

The distribution technology may change.

The dependency may be transformed or redistributed.

The requirement for evidence-supported origin and traceability remains.

DOTM does not assume that unknown provenance proves a component is malicious.

Nor does it assume that verified provenance makes a component safe, suitable, or authorized.

It establishes the evidence-supported historical relationships necessary

for competent dependency governance.

DOTM transforms software dependency provenance from a collection of source

claims and technical records into a governed chain of origin and reliance — ensuring that software systems can establish not only what they depend on, but where those dependencies came from, how they reached the system, what changed along the way, and which parts of that history can actually be proven.